

AI Usage Notes

Config Drift Detector Project

1. AI Tools Used

Tool	Purpose in This Project	Role
Claude (Anthropic)	Code generation, JSON samples, viva prep, PDF & PPT creation	Primary
ChatGPT (OpenAI)	Initial code drafts, concept lookups, cross-checking logic	Secondary
Google (Search)	Finding libraries, deployment docs, Streamlit hosting guides	Reference

2. What AI Helped With

Writing the Python / Streamlit Code

- Generated the core drift detection logic — loading JSONs, flattening nested keys using dot-notation, and comparing values.
- Built the Streamlit UI with file upload widgets, colour-coded drift tables, and the health score display.
- Wrote helper functions for the security audit pass that checks sensitive parameters like `database.ssl`, `auth.mfa_required`, and `rate_limiting.enabled`.
- Provided iterative fixes when code had bugs — paste error → get corrected version.

Generating Sample Config JSON Files

- Created realistic `config.development.json` and `config.production.json` files with 40 mapped keys across 10 sections (server, database, cache, logging, auth, feature_flags, rate_limiting, cors, email, storage, monitoring).
- Intentionally designed 35 modified keys and 5 identical keys to produce meaningful, testable drift output.
- Ensured all security-sensitive keys drifted in the correct direction (false → true for `ssl`, `mfa`, `rate_limiting`).

Explaining Concepts & Viva Preparation

- Broke down each section of the generated report (Executive Summary, Security Alerts, Drift Audit Table) into simple, team-friendly language.
- Explained why a 15% health score does not mean high risk — separating drift volume from drift severity.
- Provided the car analogy: dev car (relaxed) vs prod car (seatbelts mandatory) to make the concept memorable.
- Prepared viva one-liners and section-by-section talking points for oral presentation.

Creating the PDF Explainer & PowerPoint

- Generated a full 10-section PDF explainer using ReportLab with colour-coded tables, info boxes, and a risk matrix.
- Built a 7-slide ocean-themed PowerPoint using PptxGenJS — with layered wave shapes, bubble decorations, stat chips, and pipeline diagrams.
- Handled all layout, font sizing, and colour palette decisions automatically based on the ocean theme request.

3. Where AI Went Wrong

Not every AI response was correct. Below are the honest limitations encountered during this project, and how they were handled.

Tool: ChatGPT

Issue: Generated code that didn't run

ChatGPT produced a Streamlit snippet that imported a function (`st.experimental_data_editor`) that had been renamed/removed in the current Streamlit version. The code looked correct and complete but threw an `AttributeError` at runtime. This required cross-checking the Streamlit changelog and switching to `st.data_editor`.

■ *Lesson learned: Always run AI-generated code in your environment before trusting it. Library APIs change — AI training data has a cutoff and may suggest outdated functions.*

Tool: ChatGPT

Issue: Hallucinated a deployment approach

When asked about deploying the Streamlit app, ChatGPT suggested a configuration option that doesn't exist in the standard Streamlit Cloud settings panel. It described a feature with confidence but the option was simply not there.

■ *Lesson learned: For deployment and platform-specific steps, always verify against the official documentation. AI is better at code logic than platform UX details.*

4. Best Prompts Used

These are the most effective prompts used during the project. The prompting style was **iterative** — starting with a broad request, then refining based on the output each time.

Prompt 1 — Project Bootstrap

```
"Build a config drift detector in Python/Streamlit. It should take two JSON files (dev and prod), compare all keys using dot-notation, classify each as Modified / Identical / Added / Removed, show a health score, and run a security check on sensitive keys like ssl, mfa, and rate_limiting."
```

Why it worked: *Giving the full spec upfront — including the classification system and security audit requirement — meant the first response was 80% usable instead of generic boilerplate.*

Prompt 2 — Generating Test Data

```
"Create two realistic JSON config files – config.development.json and config.production.json – with 40 total keys across server, database, cache, logging, auth, feature_flags, storage, email, and monitoring sections. Make 35 keys different and 5 identical. Ensure all security params are correctly hardened in prod."
```

Why it worked: *Being specific about the number of keys, sections, and the security direction constraint produced data that gave a meaningful report immediately.*

Prompt 3 — Iterative Bug Fix

```
"This code throws: AttributeError: module 'streamlit' has no attribute
'experimental_data_editor'. Here is the full error traceback: [pasted traceback].
Fix it for the current version of Streamlit."
```

Why it worked: Pasting the exact error traceback instead of describing the problem vaguely got a precise, working fix in one turn.

Prompt 4 — Viva Explanation

```
"Explain each section of this config drift report to me as if I'm presenting it to
my college team in a viva. Keep it conversational, not too technical. Tell me what
to say for the 15% health score, the zero security alerts, and the drift table."
```

Why it worked: Specifying the audience (college viva), the tone (conversational), and the exact sections to cover produced ready-to-use talking points instead of a generic explanation.

Prompt 5 — Presentation Creation

```
"Create a 7-slide PowerPoint for this project in a minimal ocean theme. Use deep
navy and teal colours. Include: title slide with stat chips, what is config drift,
how detector works, dev vs prod side by side, drift categories, health score vs
risk, and key takeaways."
```

Why it worked: Listing every slide topic explicitly and naming the visual style (minimal ocean) prevented generic slides and gave a structured, consistent deck in one pass.

5. Claude vs ChatGPT — Which Performed Better?

Criteria	Claude	ChatGPT
Code correctness	■ Ran first time mostly	■ Sometimes outdated APIs
Explanation quality	■ Clear, layered, viva-ready	■ Good but more generic
Long-form document output	■ PDF & PPT built in one go	■ Limited file creation
Iterative refinement	■ Remembered full context	■ Also good iteratively
Hallucination rate	■ Low in this project	■ Occasional outdated info

Deployment help

■ Accurate, docs-referenced

■ Suggested non-existent option

Verdict:

Claude was the primary and better-performing tool for this project — especially for long-form output generation (PDF, PPT) and code that ran correctly on the first attempt. ChatGPT was useful for quick lookups and initial drafts but required more manual verification.

AI was used as a development assistant throughout this project. All outputs were reviewed, tested, and corrected by the student. Final understanding of the project concepts is independent of AI assistance.